S&DS 365 / 665

Intermediate Machine Learning

Reinforcement Learning: Policy Gradient Methods

November 9

Yale

Version of 2026/11/09 at 12:54

IML business

- Assignment 4 is out (GNNs and Deep RL)
- Quiz 4 Wednesday, end of class
- Project proposals due Thursday

Outline for today

- Automatic differentiation
- Bandit problems
- Policy methods: Grad-log trick
- Simple demo
- Improvements to policy gradients

Automatic differentiation

- For supervised problems, loss function is $L(\widehat{Y}, Y)$
- Can program this directly
- RL loss functions are built up dynamically as agent makes decisions
- Automatic differentiation allows us to handle this

`https://www.tensorflow.org/guide/autodiff`

`https://pytorch.org/tutorials/beginner/basics/autogradqs_tutorial.html`

Automatic differentiation: Gradient collection

Modern platforms support automatic differentiation by recording relevant operations executed inside the context of a “tape”

This record is then used to compute the numerical values of gradients using “reverse mode differentiation”

---

https://en.wikipedia.org/wiki/Automatic_differentiation

Automatic differentiation: Hello world!

```
import numpy as np
import torch
```

0.5s Python

```
x = torch.tensor(3.0, requires_grad=True)

y = x**2

y.backward()
dy_dx = x.grad
print(dy_dx.numpy())
```

0.0s Python

```
6.0
```

https://pytorch.org/tutorials/beginner/basics/autogradqs_tutorial.html

Automatic differentiation: Hello world!

```
w = torch.randn(3, 2, requires_grad=True)
b = torch.zeros(2, dtype=torch.float32, requires_grad=True)
x = torch.tensor([[1., 2., 3.]])

y = x @ w + b
loss = torch.mean(y**2)

[dloss_dw, dloss_db] = torch.autograd.grad(loss, [w, b])
print(dloss_dw.numpy(), "\n\n", dloss_db.numpy())
```

[3] ✓ 0.0s Python

```
[[ 0.63157207 -0.3194919 ]
 [ 1.2631441  -0.6389838 ]
 [ 1.8947163  -0.9584757 ]]

 [ 0.63157207 -0.3194919 ]
```

https://pytorch.org/tutorials/beginner/basics/autogradqs_tutorial.html

Multi-armed bandits



Multi-armed bandits

- The rewards are independent and noisy
- Arm k has expected payoff μ_k with variance σ_k^2 on each pull
- Each time step, pull an arm and observe the resulting reward
- Played often enough, can estimate mean reward of each arm
- What is the best policy?
- Exploration-exploitation tradeoff
- “Contextual bandits” add covariate vector — but actions don’t impact the environment

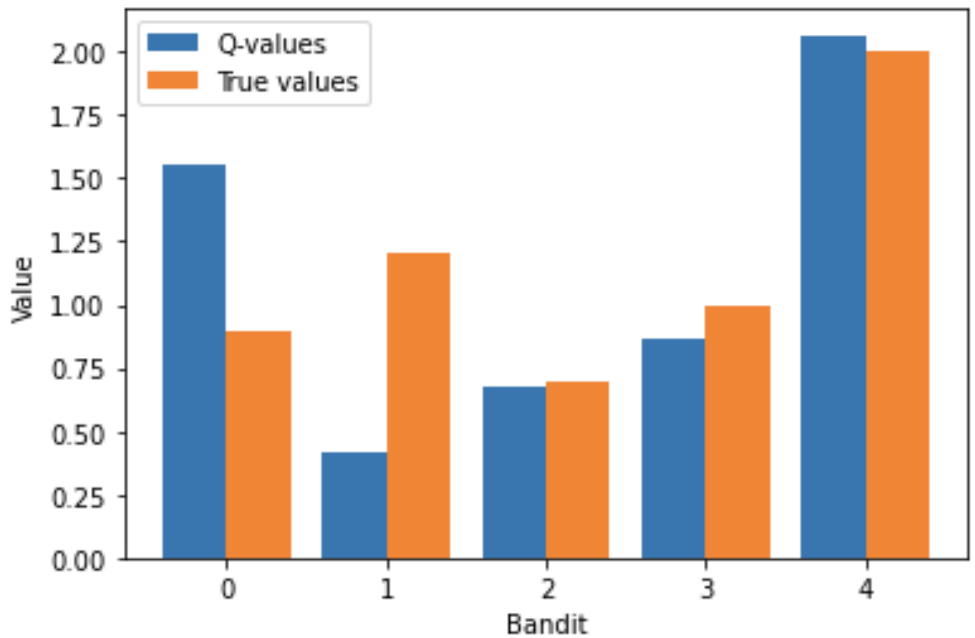
Multi-armed bandits

We'll treat this as an RL problem and hit it with a big hammer: Deep Q-learning

Multi-armed bandits

```
======episode 10000 ======
Q-values ['1.556', '0.412', '0.675', '0.866', '2.065']
Deviation ['72.8%', '-65.7%', '-3.6%', '-13.4%', '3.3%']

<Figure size 864x504 with 0 Axes>
```

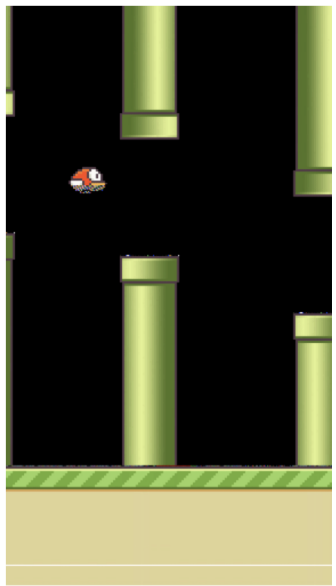


Let's go to the notebook!

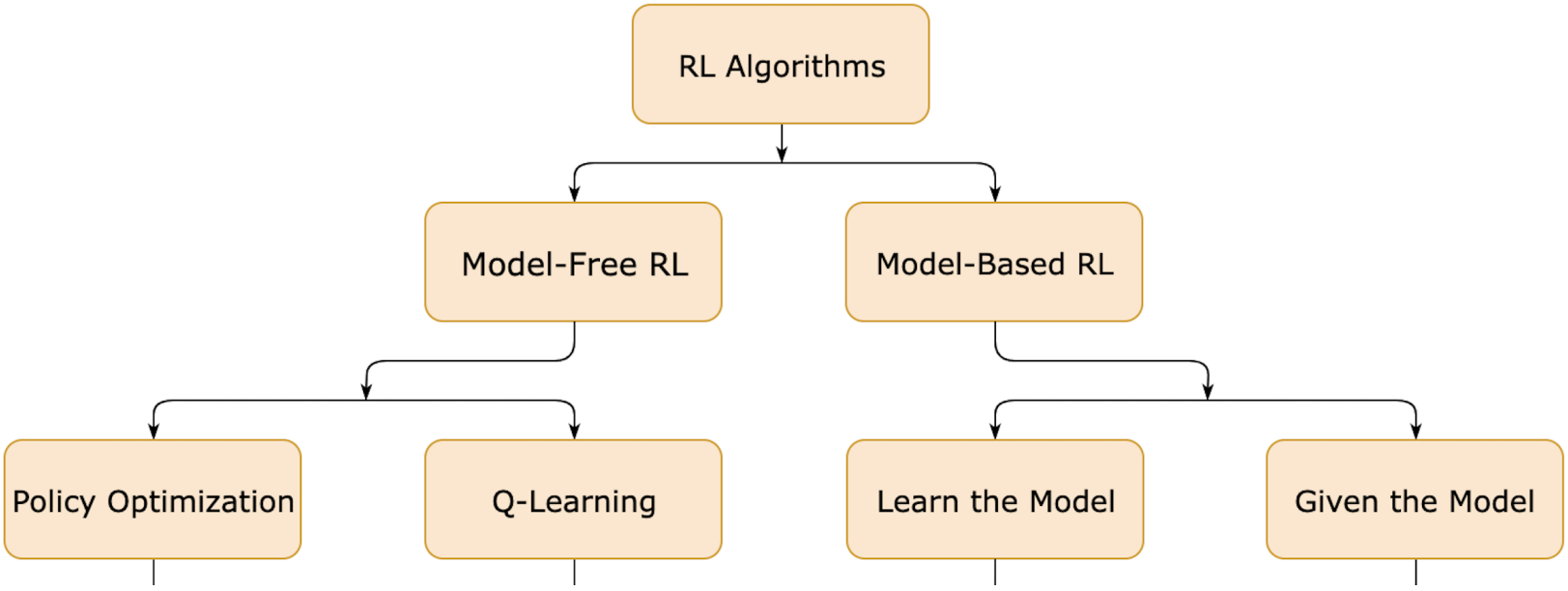
Assn 4: Flappy Bird

Problem 3: Deep Q-Learning for Flappy Bird (25 points)

In this problem, we will walk you through the implementation of deep Q learning to learn to play the Flappy Bird game.



Landscape of RL algorithms



https://tinyurl.com/25vruwtp

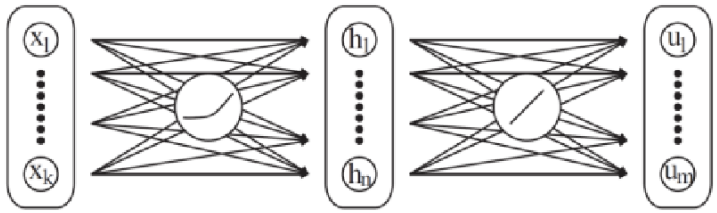
Q-learning vs. Policy methods

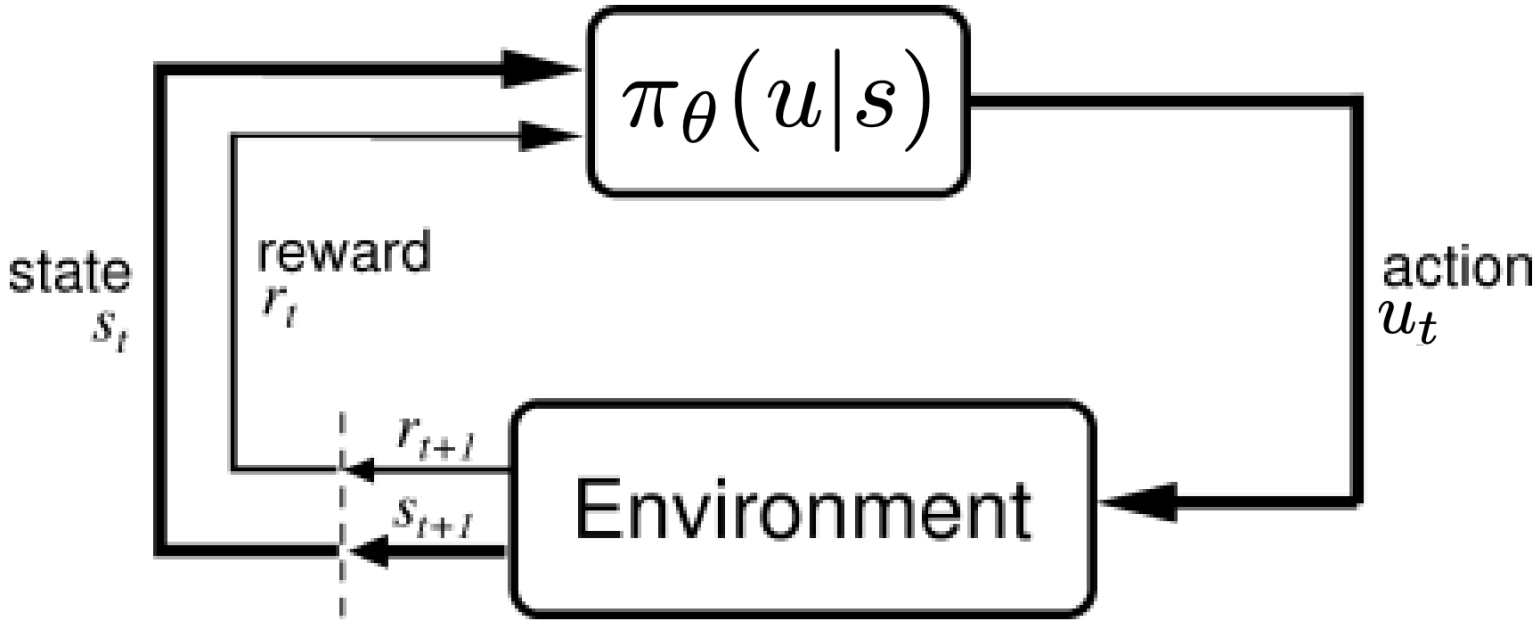
- Q-learning targets Bellman equation
- Policy methods target loss (reward)

Policy gradient methods

- policy $\pi_\theta(s)$ has parameters θ
- Perform gradient ascent over θ
- Well-suited to deep learning approaches
- Why use an on-policy method? May be possible to estimate a good policy without accurately estimating the value function or Q-function
- Can be more compute-efficient and stable than Q-learning

Policy gradient methods





Successes of policy methods



Kohl and Stone, 2004



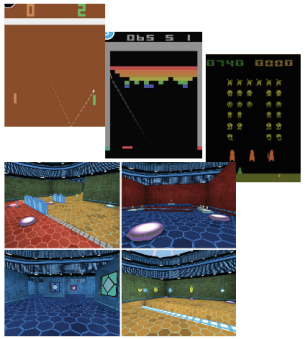
Ng et al, 2004



Tedrake et al, 2005



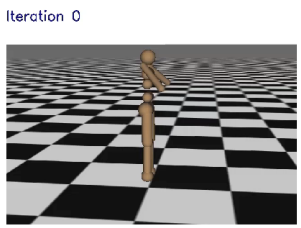
Kober and Peters, 2009



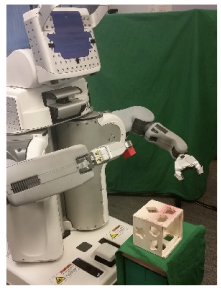
Mnih et al, 2015
(A3C)



Silver et al, 2014
(DPG)
Lillicrap et al, 2015
(DDPG)



Schulman et al,
2016 (TRPO + GAE)



Levine*, Finn*, et
al, 2016
(GPS)



Silver*, Huang*, et
al, 2016
(AlphaGo**)

Policy gradient methods: Loss function

We start with the loss function: Expected reward $\mathcal{J}(\theta) = \mathbb{E}(R)$

- Policy $\pi(s; \theta)$ has parameters θ
- Perform gradient ascent on $\mathcal{J}(\theta)$
- Well-suited to deep learning approaches
- Neural network policy extracts predictive "features" of states

We'll need to develop some mathematics to understand how policy methods can be efficiently applied...

Policy gradient methods: Loss function

Policy is probability distribution $\pi_\theta(a \mid s)$ over actions given state s.

The episode unfolds as a random sequence τ

$$\tau : (s_0, a_0) \to (s_1, r_1, a_1) \to (s_2, r_2, a_2) \to \cdots \to (s_T, r_T, a_T) \to s_{T+1}$$

where s_{T+1} is a terminal state. Receive reward $R(\tau)$, for example

$$R(\tau) = \sum_{t=1}^{T} r_t$$

Objective function $\mathcal{J}$ is expected reward

$$\mathcal{J}(\theta) = \mathbb{E}_\theta(R(\tau))$$

Calculating the gradient

Using Markov property, calculate $\mathbb{E}_\theta(R(\tau))$ as

$$\mathbb{E}_\theta(R(\tau)) = \int p(\tau \mid \theta) R(\tau)\, d\tau$$

$$p(\tau \mid \theta) = \prod_{t=0}^{T} \pi_\theta(a_t \mid s_t)\, p(s_{t+1}, r_{t+1} \mid s_t, a_t)$$

If states or rewards are finite the integral becomes a sum, or a mix of sums and integrals.

Calculating the gradient

Using Markov property, calculate $\mathbb{E}_\theta(R(\tau))$ as

$$\mathbb{E}_\theta(R(\tau)) = \int p(\tau \mid \theta) R(\tau)\, d\tau$$

$$p(\tau \mid \theta) = \prod_{t=0}^{T} \pi_\theta(a_t \mid s_t)\, p(s_{t+1}, r_{t+1} \mid s_t, a_t)$$

It follows that

$$\nabla_\theta \log p(\tau \mid \theta) = \sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t \mid s_t) = \sum_{t=0}^{T} \frac{\nabla_\theta \pi_\theta(a_t \mid s_t)}{\pi_\theta(a_t \mid s_t)}$$

If states or rewards are finite the integral becomes a sum, or a mix of sums and integrals.

Calculating the gradient

Now we use

$$
\begin{aligned}
\nabla_\theta \mathcal{J}(\theta) &= \nabla_\theta \, \mathbb{E}_\theta R(\tau) \\
&= \nabla_\theta \int R(\tau) \; p(\tau \,|\, \theta) \; d\tau \\
&= \int R(\tau) \, \nabla_\theta p(\tau \,|\, \theta) \; d\tau \\
&= \int R(\tau) \, \frac{\nabla_\theta p(\tau \,|\, \theta)}{p(\tau \,|\, \theta)} \; p(\tau \,|\, \theta) \; d\tau \\
&= \mathbb{E}_\theta \Big(R(\tau) \nabla_\theta \log p(\tau \,|\, \theta) \Big)
\end{aligned}
$$

Need regularity conditions to exchange derivative and integral, such as uniform continuity of the derivatives of the policy.

Approximating the gradient

Since it's now an expectation, can approximate by sampling:

$$\begin{aligned}
\nabla_\theta \mathcal{J}(\theta) &\approx \frac{1}{N} \sum_{i=1}^{N} R(\tau^{(i)}) \nabla_\theta \log p(\tau^{(i)} \mid \theta) \\
&= \frac{1}{N} \sum_{i=1}^{N} R(\tau^{(i)}) \sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t^{(i)} \mid s_t^{(i)}) \\
&\equiv \widehat{\nabla_\theta \mathcal{J}(\theta)}
\end{aligned}$$

Approximating the gradient

Since it's now an expectation, can approximate by sampling:

$$\begin{aligned}
\nabla_\theta \mathcal{J}(\theta) &\approx \frac{1}{N} \sum_{i=1}^{N} R(\tau^{(i)}) \nabla_\theta \log p(\tau^{(i)} \mid \theta) \\
&= \frac{1}{N} \sum_{i=1}^{N} R(\tau^{(i)}) \sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t^{(i)} \mid s_t^{(i)}) \\
&\equiv \widehat{\nabla_\theta \mathcal{J}(\theta)}
\end{aligned}$$

$\tau^{(i)}$ is sometimes called a "rollout" of the policy π_θ

Approximating the gradient

Since it's now an expectation, can approximate by sampling:

$$\begin{aligned}\nabla_\theta \mathcal{J}(\theta) &\approx \frac{1}{N}\sum_{i=1}^{N} R(\tau^{(i)})\nabla_\theta \log p(\tau^{(i)} \mid \theta) \\ &= \frac{1}{N}\sum_{i=1}^{N} R(\tau^{(i)}) \sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t^{(i)} \mid s_t^{(i)}) \\ &\equiv \widehat{\nabla_\theta \mathcal{J}(\theta)}\end{aligned}$$

$\tau^{(i)}$ is sometimes called a "rollout" of the policy π_θ

The policy gradient algorithm is then

$$\theta \longleftarrow \theta + \eta \widehat{\nabla_\theta \mathcal{J}(\theta)}$$

Intuition

Gradient ascent of the expected reward with respect to policy:

- Increases probability of paths τ with large R
- Decreases probability of paths τ with small R

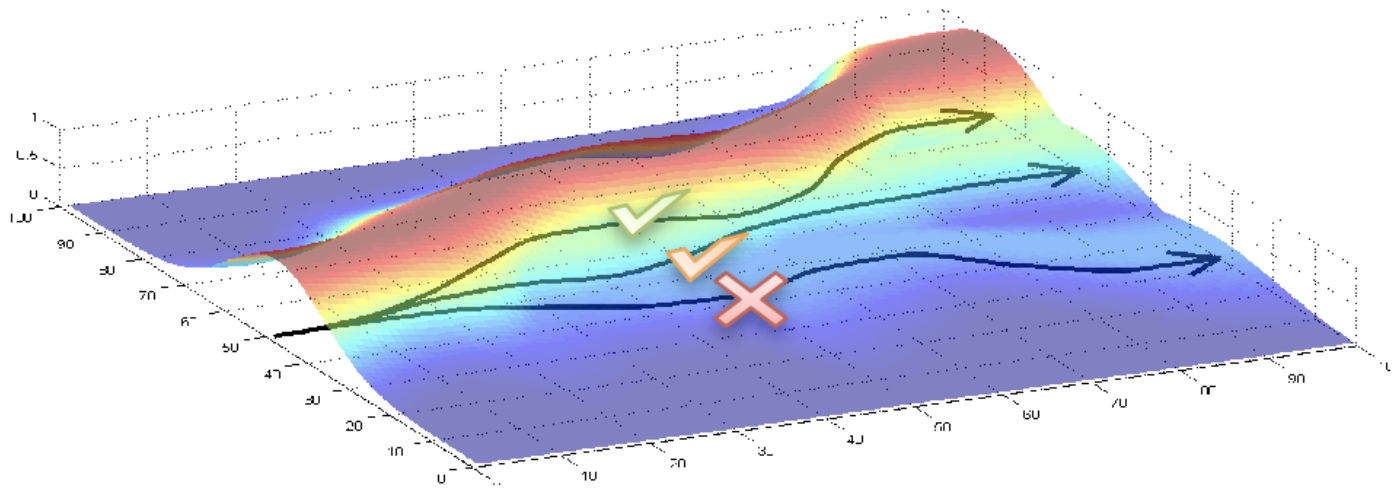
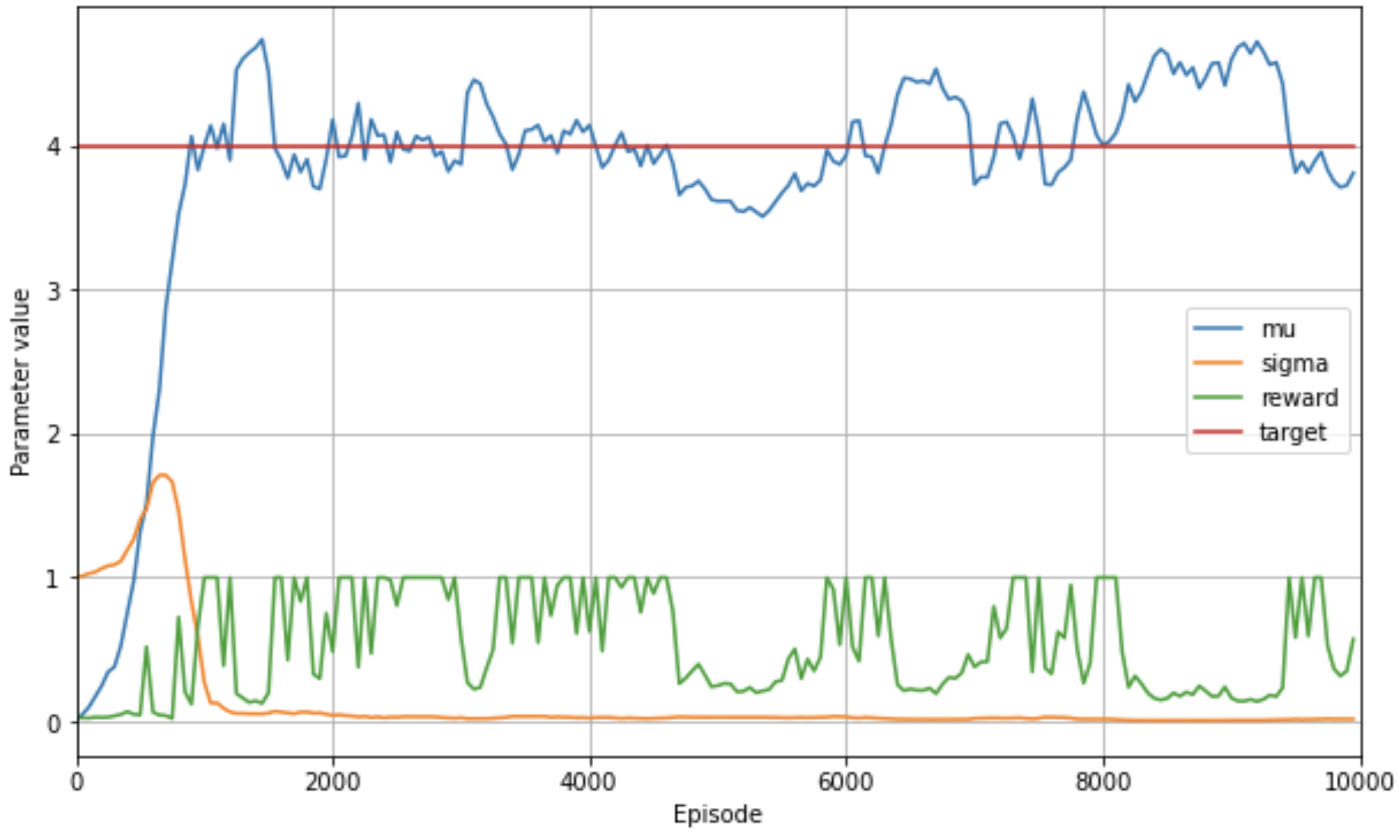


Fig from Schulman and Abbeel

Simple example



Simple example



Let’s go to the notebook

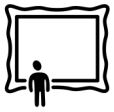
Summary so far

- Policy methods estimate $\pi(a \mid s)$
- Tabular methods can be used for small state/action spaces
- Otherwise, parameterize $\pi_\theta(a \mid s)$ and use gradient ascent
  - Change in expected reward calculated with “grad-log trick”

The key equation

We showed via the "grad-log trick" and $\log(\pi_\theta \cdot p) = \log \pi_\theta + \log p$ that

$$\nabla_\theta \mathcal{J}(\theta) = \mathbb{E}_\theta \left(R(\tau) \sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t \mid s_t) \right)$$



Why is this important?

- This manipulation is important because it gets the policy explicity into the objective function as an expectation
- Similar to the “reparameterization trick” for VAEs

How to make this more efficient?

- If rewards are all positive, we might increase weight on all actions — with larger increases when rewards are greater
- We actually want to boost up actions associated with rewards that have *greater than average* reward
- Similarly, downweight actions associated with rewards that have below average reward
- This can *reduce the variance* of policy estimates

Improvements

The following are some improvements to policy gradient methods

- Important in practice
- But not conceptually, for this class
- You're not responsible for following details

Baseline subtraction

Our “rollout” gradient

$$\nabla_\theta \mathcal{J}(\theta) \approx \frac{1}{N} \sum_{i=1}^{N} \nabla_\theta \log p(\tau^{(i)} \mid \theta) R(\tau^{(i)})$$

is an unbiased estimate of the true model gradient:

$$\mathbb{E}\left(\frac{1}{N} \sum_{i=1}^{N} \nabla_\theta \log p(\tau^{(i)} \mid \theta) R(\tau^{(i)}) \right) = \nabla_\theta \mathcal{J}(\theta)$$

Baseline subtraction

But then

$$\nabla_\theta \mathcal{J}(\theta) \approx \frac{1}{N} \sum_{i=1}^{N} \nabla_\theta \log p(\tau^{(i)} \mid \theta) \left(R(\tau^{(i)}) - b \right)$$

is *also* an unbiased estimate of the gradient, where b is a "baseline" average.

Why?

Baseline subtraction

This is because

$$\begin{aligned}
\mathbb{E}\left(\nabla_\theta \log p(\tau \mid \theta)\, b\right) &= \sum_\tau p(\tau \mid \theta) \nabla_\theta \log p(\tau \mid \theta)\, b \\
&= \sum_\tau \nabla_\theta p(\tau \mid \theta)\, b \\
&= \nabla_\theta \sum_\tau p(\tau \mid \theta)\, b \\
&= b \nabla_\theta \sum_\tau p(\tau \mid \theta) \\
&= b \nabla_\theta 1 \\
&= 0
\end{aligned}$$

Same result will hold as long as b doesn't depend on the actions a_t

Introducing time dependence

Now we can refine this. The gradient is

$$\nabla_\theta \mathcal{J}(\theta) \approx \frac{1}{N} \sum_{i=1}^{N} \nabla_\theta \log p(\tau^{(i)} \mid \theta)(R(\tau^{(i)}) - b)$$

$$= \frac{1}{N} \sum_{i=1}^{N} \left(\sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t^{(i)} \mid s_t^{(i)}) \right) \left(\sum_{t=0}^{T} r_t^{(i)} - b \right)$$

$$= \frac{1}{N} \sum_{i=1}^{N} \left(\sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t^{(i)} \mid s_t^{(i)}) \left(\sum_{k=0}^{t} r_k^{(i)} + \sum_{k=t+1}^{T} r_k^{(i)} - b \right) \right)$$

But action a_t only impacts rewards after time t.

Here we're using the reward at step k as $r_k = \text{reward}(s_k, a_k)$

Introducing time dependence

So, we replace this with

$$\nabla_\theta \mathcal{J}(\theta) \approx \frac{1}{N} \sum_{i=1}^{N} \sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t^{(i)} \mid s_t^{(i)}) \left(\sum_{k=t+1}^{T} r_k^{(i)} - b \right)$$

or

$$\nabla_\theta \mathcal{J}(\theta) \approx \frac{1}{N} \sum_{i=1}^{N} \sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t^{(i)} \mid s_t^{(i)}) \left(\sum_{k=t+1}^{T} r_k^{(i)} - b(s_t^{(i)}) \right)$$

We weight the gradients at time t by the total reward from that time onwards minus the average reward

This makes more efficient use of the samples.

Estimating b

We use the improved (lower variance) gradient estimate

$$\nabla_\theta \mathcal{J}(\theta) \approx \frac{1}{N} \sum_{i=1}^{N} \sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t^{(i)} \mid s_t^{(i)}) \left(\sum_{k=t+1}^{T} r_k^{(i)} - b(s_t^{(i)}) \right)$$

Estimate b with regression using another neural network with parameters ϕ:

$$\phi \leftarrow \arg\min_{\phi} \frac{1}{N} \sum_{i=1}^{N} \sum_{t=0}^{T} \left(\sum_{k=t+1}^{T} r_k^{(i)} - b_\phi(s_t^{(i)}) \right)^2$$

Using the value function

But the average discounted reward starting from the state s_t is the *value function*:

$$b(s_t) = \mathbb{E}(r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \cdots + \gamma^{T-t} r_T) = V^\pi(s_t)$$

So, we are led to simultaneously look for an estimate of the value function while estimating the policy.

This is a type of RL called *actor-critic reinforcement learning*.

Using the Bellman equation

- Roll out policy, collecting paths $\tau^{(i)}$ and instances of (s, a, s_{next}, r)
- Update value function:

$$\phi \leftarrow \arg\min_{\phi} \sum_{(s,a,s_{next},r)} \left(r + \gamma V^{\pi}_{\phi_{old}}(s_{next}) - V^{\pi}_{\phi}(s) \right)^2 + \lambda \|\phi - \phi_{old}\|^2$$

- Update policy:

$$\theta \leftarrow \theta + \eta \frac{1}{N} \sum_{i=1}^{N} \sum_{t=0}^{T} \nabla_{\theta} \log \pi(a_t^{(i)} \mid s_t^{(i)}) \underbrace{\left(\sum_{k=t+1}^{T} \gamma^{k-t-1} r_k^{(i)} - V^{\pi}_{\phi}(s_t^{(i)}) \right)}_{\textit{Advantage}}$$

Summary

- Policy gradients have a nice form
- Can reduce variance in gradient ascent by comparing to baseline
- Natural baseline is given by the value function
- Value function can be estimated during policy estimation
- Modern RL methods develop different refinements to this approach